

ADITYA UNIVERSITY

Surampalem

Department of AIML

DBMS CAPSTONE PROJECT

Project Documentation Report

Project Title: E-Commerce Order Management System.

Submitted by:

S.No	Student Name	Roll Number
1	K. Poojitha	25B11AI545
2	V. Chandrika	25B11AIC65
3	K. Gayathri	25B11AI624
4	K. Jeevan	25B11AI553

Course: Database Management Systems (DBMS) Course Code: 25B

Faculty Coordinator: T. Srinivasulu

Academic Year: 2026 Date of Submission: -08-2026.

GitHub Repository Link: <https://github.com/poojithakolla-sketch/ecommerce-order-management-system>

TABLE OF CONTENTS

(This table updates automatically when the document is opened in Microsoft Word. If it appears empty, right-click it and choose "Update Field".)

TABLE OF CONTENTS	1
1. ABSTRACT	1
2. INTRODUCTION	1
2.1 Background	1
2.2 Purpose of the Database	1
3. PROBLEM STATEMENT	1
4. OBJECTIVES	1
5. TOOLS & TECHNOLOGIES USED	1
6. DATABASE DESIGN	1
6.1 Identified Entities	1
6.2 Entity Attributes	1
6.3 ER Diagram	1
6.4 Relationships & Cardinality	1
6.5 Relational Database Schema	1
6.6 Normalization	1
7. DATABASE IMPLEMENTATION	1
7.1 DDL — Database and Table Creation	1
7.2 Sample Data — DML	1
7.3 Constraints & Data Integrity	1
7.4 SQL Operations & Queries	1
7.5 Sample Outputs / Screenshots	1
8. MODULES & FUNCTIONALITIES	1
9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES	1
10. ENHANCEMENTS & FUTURE IMPROVEMENTS	1
11. CONCLUSION	1
12. REFERENCES	1
APPENDIX	1
A. Complete SQL Script	1
B. Additional Screenshots / Outputs	1

1. ABSTRACT

- The **E-Commerce Order Management System** is a database-based system designed to manage customers, products, orders, payments, and deliveries efficiently.
 - The system stores and organizes order-related information in a structured database. It helps track the complete order lifecycle, from placing an order to payment and delivery.
 - The database reduces data redundancy, improves data consistency, and makes order information easier to retrieve and manage.
 - The system enables quick retrieval of order information and supports efficient monitoring of order status from placement to delivery.
-

2. INTRODUCTION

2.1 Background

- The **E-Commerce Order Management System** is a database-based system designed to manage customers, products, orders, payments, and deliveries efficiently.
 - The system stores and organizes order-related information in a structured database. It helps track the complete order lifecycle, from placing an order to payment and delivery.
 - The database reduces data redundancy, improves data consistency, and makes order information easier to retrieve and manage.
 - The system enables quick retrieval of order information and supports efficient monitoring of order status from placement to delivery.
-

2.2 Purpose of the Database

- It helps improve **order accuracy, data consistency, and efficient access to information** for better order management.

3. PROBLEM STATEMENT

Existing Problems:

- Difficulty in managing large numbers of customer orders.
- Duplicate or inconsistent customer and product information.
- Difficult to track order status and delivery details.
- Payment information needs proper organization.
- Generating order and sales reports can be difficult.

Proposed Solution:

Develop a relational database that connects customers, products, orders, payments, and deliveries to provide efficient order management.

4. OBJECTIVE

- Objective 1: **Manage Customer Information**
Maintain customer details such as customer ID, name, email, phone number, and address in a structured manner.
- Objective 2: **Manage Customer Information**
Maintain customer details such as customer ID, name, email, phone number, and address in a structured manner.
- Objective 3: **_Manage Customer Information**
Maintain customer details such as customer ID, name, email, phone number, and address in a structured manner.
- Objective 4: **__Track Payments and Deliveries**
Maintain payment details and delivery information associated with each order.

- Objective 5: **_. Maintain Data Integrity**

Use primary keys and foreign keys to establish relationships between tables and prevent inconsistent data.

5. TOOLS & TECHNOLOGIES USED

List the DBMS software, IDE/client tool, and any front-end or scripting technology used to build and demonstrate the project.

Category	Tool / Technology Used
Database Server	Oracle Database
Query / Admin Tool	Oracle SQL Developer
Front-end (if any)	Not Used
Operating System	Windows 11
Version Control	GitHub

6. DATABASE DESIGN

6.1 Identified Entities

Minimum: 6 entities. Provide a short description of each entity.

S. No	Entity Name	Description
1	Customer	Stores customer identity and contact details.
2	Category	Groups products into logical categories.
3	Product	Stores product information, price, and stock.
4	Orders	Records a customer's order and its overall status.
5	Payment	Records payment method, amount, date, and status.
6	Order_item	Stores delivery address, date, and status.
7	Address	Stores customer delivery address details such as street, city, state, and pincode.

6.2 Entity Attributes

List the attributes of each entity. Mark Primary Keys (PK) and Foreign Keys (FK).

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Customer	customer_id	INT	PK	Simple
Customer	first_name	VARCHAR(50)	—	Composite
Customer	last_name	VARCHAR(50)	—	Composite
Customer	email	VARCHAR(100)	—	Simple
Customer	phone_numbers	TEXT	—	Multi-valued
Customer	registration_date	DATE	—	Simple
Customer	total_orders	INT	—	Derived
Address	address_id	INT	PK	Simple
Address	customer_id	INT	FK	Simple
Address	street	VARCHAR(200)	—	Composite

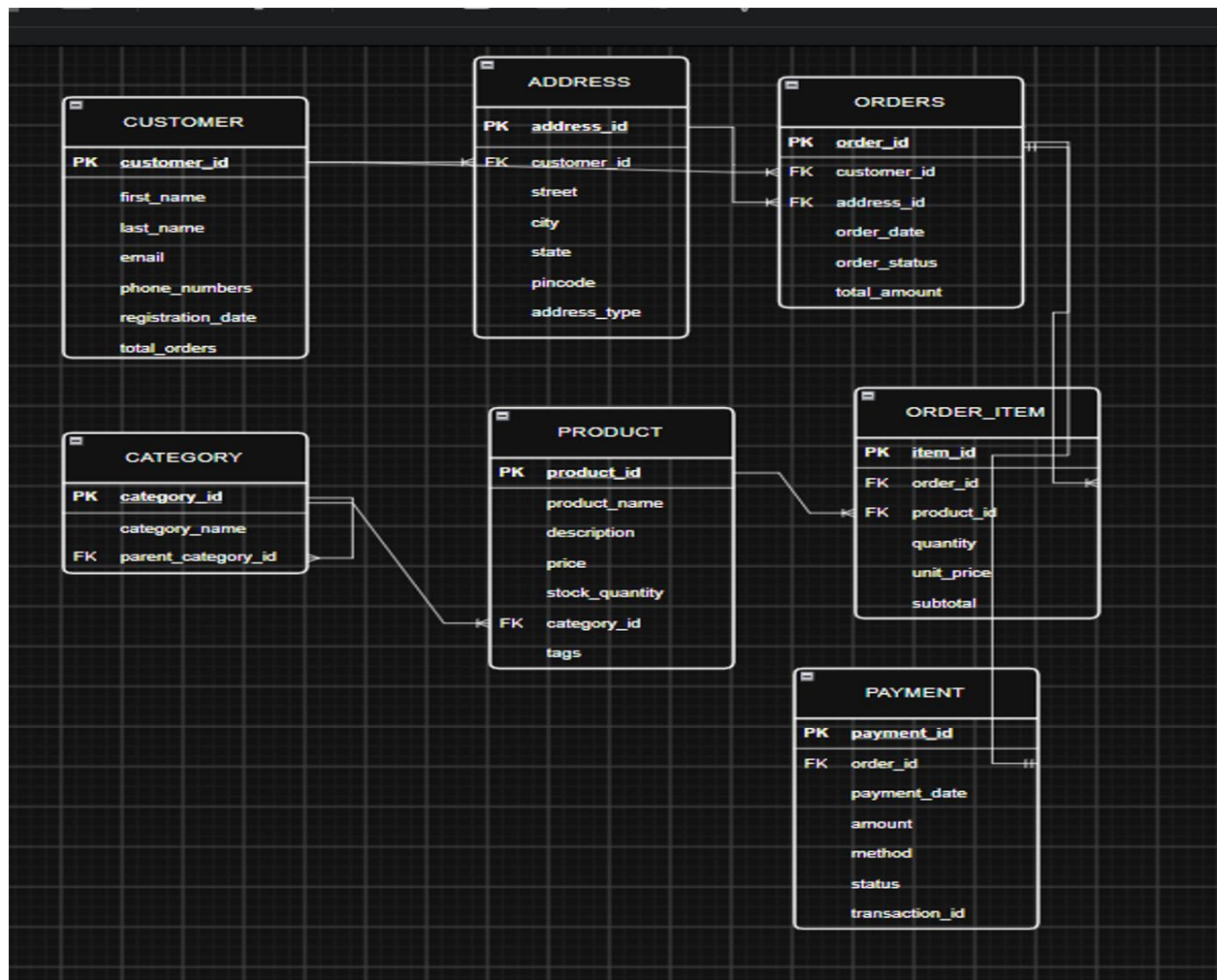
Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Address	city	VARCHAR(100)	—	Composite
Address	state	VARCHAR(100)	—	Composite
Address	pincode	VARCHAR(10)	—	Composite
Address	address_type	VARCHAR(20)	—	Simple
Category	category_id	INT	PK	Simple
Category	category_name	VARCHAR(100)	—	Simple
Category	parent_category_id	INT	FK	Simple
Product	product_id	INT	PK	Simple
Product	product_name	VARCHAR(200)	—	Simple
Product	description	TEXT	—	Simple
Product	price	DECIMAL(10,2)	—	Simple
Product	stock_quantity	INT	—	Simple
Product	category_id	INT	FK	Simple
Product	tags	TEXT	—	Multi-valued
Orders	order_id	INT	PK	Simple
Orders	customer_id	INT	FK	Simple
Orders	address_id	INT	FK	Simple
Orders	order_date	DATE	—	Simple
Orders	order_status	VARCHAR(30)	—	Simple
Orders	Total_amount	DECIMAL(10,2)	—	Derived
Order_Item	Item_id	INT	PK	Simple
Order_Item	Order_id	INT	FK	Simple
Order_Item	Product_id	INT	FK	Simple
Order_Item	Quantity	INT	—	Simple

Entity	Attribute	Data Type	Key (PK/FK/—)	Attribute Type
Order_Item	Unit_price	DECIMAL(10,2)	—	Simple
Order_Item	Subtotal	DECIMAL(10,2)	—	Derived
Payment	Payment_id	INT	PK	Simple
Payment	Order_id	INT	FK	Simple
Payment	Payment_date	DATE	—	Simple
Payment	Amount	DECIMAL(10,2)	—	Simple
Payment	Method	VARCHAR(30)	—	Simple
Payment	Status	VARCHAR(20)	—	Simple
Payment	Transaction_id	VARCHAR(100)	—	Simple

Attribute Type: Simple / Composite / Multi-valued / Derived

6.3 ER Diagram

Insert the complete ER diagram showing entities, attributes, relationships, and cardinalities.



6.4 Relationships & Cardinality:

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)
Customer	has	Address	1 : N
Customer	places	Orders	1 : N
Address	Is used for	Orders	1:N
Category	Contains	Product	1:N

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)
Orders	Contains	Order_Item	1:N
Product	Appears in	Order_Item	1:N
Orders	has	Payment	1:N
Category	Has parent category	Category	1:N

6.5 Relational Database Schema

List every table, its columns, data types, and identify PK/FK columns.

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Customer	customer_id	INT	PK	AUTO_INCREMENT, NOT NULL
Customer	first_name	VARCHAR(50)	—	NOT NULL
Customer	last_name	VARCHAR(50)	—	NOT NULL
Customer	email	VARCHAR(100)	—	NOT NULL, UNIQUE
Customer	phone_numbers	TEXT	—	—
Customer	registration_date	DATE	—	DEFAULT CURRENT_DATE
Customer	total_orders	INT	—	Derived
Address	address_id	INT	PK	AUTO_INCREMENT, NOT NULL
Address	customer_id	INT	FK	REFERENCES Customer(customer_id)
Address	street	VARCHAR(200)	—	NOT NULL
Address	city	VARCHAR(100)	—	NOT NULL
Address	state	VARCHAR(100)	—	NOT NULL
Address	pincode	VARCHAR(10)	—	NOT NULL
Address	address_type	VARCHAR(20)	—	—
Category	category_id	INT	PK	AUTO_INCREMENT, NOT NULL
Category	category_name	VARCHAR(100)	—	NOT NULL, UNIQUE
Category	parent_category_id	INT	FK	REFERENCES Category(category_id)
Product	product_id	INT	PK	AUTO_INCREMENT, NOT NULL
Product	product_name	VARCHAR(200)	—	NOT NULL
Product	description	TEXT	—	—
Product	price	DECIMAL(10,2)	—	NOT NULL, CHECK (price >= 0)
Product	stock_quantity	INT	—	—

Table Name	Column Name	Data Type	PK/FK/—	Constraints
Product	category_id	INT	FK	REFERENCES Category(category_id)
Product	tags	TEXT	—	—
Orders	order_id	INT	PK	AUTO_INCREMENT, NOT NULL
Orders	customer_id	INT	FK	REFERENCES Customer(customer_id)
Orders	address_id	INT	FK	REFERENCES Address(address_id)
Orders	order_date	DATE	—	—
Orders	Order_status	VARCHAR(30)	—	DEFAULT 'Pending'
Orders	Total_amount	DECIMAL(10,2)	—	Derived
Order_item	Item_id	INT	PK	AUTO_INCREMENT, NOT NULL
Order_item	Order_id	INT	FK	REFERENCES Orders(order_id)
Order_item	Product_id	INT	FK	REFERENCES Product(product_id)
Order_item	Quantity	INT	—	NOT NULL, CHECK (quantity > 0)
Order_item	Unit_price	DECIMAL(10,2)	—	—
Order_item	Subtotal	DECIMAL(10,2)	—	Derived
Payment	Payment_id	INT	PK	AUTO_INCREMENT, NOT NULL
Payment	Order_id	INT	FK	REFERENCES Orders(order_id)
Payment	Payment_date	DATE	—	—
Payment	Amount	DECIMAL(10,2)	—	—
Payment	Method	VARCHAR(30)	—	—
Payment	Status	VARCHAR(20)	—	DEFAULT 'Pending'
Payment	Transaction_id	VARCHAR(100)	—	UNIQUE

6.6 Normalization

1NF: Phone numbers in Customer_Phone; tags in Product_Tags; all cells atomic.

2NF: Order_Item separates product quantity from order — no partial dependency.

3NF: unit_price stored in Order_Item (not computed from Product.price) to preserve historical accuracy; total_amount and subtotal are derived views.

7. DATABASE IMPLEMENTATION

7.1 DDL — Database and Table Creation

Paste *CREATE DATABASE / CREATE TABLE* statements and include constraints such as *NOT NULL*, *UNIQUE*, *CHECK*, *DEFAULT*, *PRIMARY KEY*, and *FOREIGN KEY*.

```
CREATE TABLE Customer (  
    customer_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    registration_date DATE DEFAULT CURRENT_DATE  
);  
  
CREATE TABLE Address (  
    address_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id INT NOT NULL,  
    street VARCHAR(200),  
    city VARCHAR(100),  
    state VARCHAR(100),  
    pincode VARCHAR(10),  
    address_type VARCHAR(20) DEFAULT 'Home',  
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);  
  
CREATE TABLE Category (  
    category_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    category_name VARCHAR(100) NOT NULL UNIQUE,  
    parent_category_id INT,  
    FOREIGN KEY (parent_category_id) REFERENCES Category(category_id)  
);
```

```
CREATE TABLE Product (  
product_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
product_name VARCHAR(200) NOT NULL,  
description TEXT,  
price DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
stock_quantity INT DEFAULT 0,  
category_id INT,  
FOREIGN KEY (category_id) REFERENCES Category(category_id)  
);  
  
CREATE TABLE Orders (  
order_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
customer_id INT NOT NULL,  
address_id INT,  
order_date DATE DEFAULT CURRENT_DATE,  
order_status VARCHAR(30) DEFAULT 'Pending',  
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
FOREIGN KEY (address_id) REFERENCES Address(address_id)  
);  
  
CREATE TABLE Order_Item (  
item_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
order_id INT NOT NULL,  
product_id INT NOT NULL,  
quantity INT NOT NULL CHECK (quantity > 0),  
unit_price DECIMAL(10,2) NOT NULL,  
FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
FOREIGN KEY (product_id) REFERENCES Product(product_id)  
);  
  
CREATE TABLE Payment (  
payment_id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```

        order_id INT UNIQUE NOT NULL,

        payment_date DATE,

        amount DECIMAL(10,2),

        method VARCHAR(30),

        status VARCHAR(20) DEFAULT 'Pending',

        transaction_id VARCHAR(100) UNIQUE,

        FOREIGN KEY (order_id) REFERENCES Orders(order_id)

    );

```

7.2 Sample Data — DML

Insert sufficient sample data into all tables. Include *INSERT*, *UPDATE*, and *DELETE* examples where applicable.

Table	No. of Records	Sample Data / Screenshot Reference
Customer	3	Anita Patel, Ramesh Nair, Divya Singh
Address	3	Hyderabad, Chennai, Bangalore addresses
Category	3	Electronics, Clothing, Mobile Phones
Product	3	Samsung Galaxy S24, Laptop HP Pavilion, Cotton T-Shirt
Order	3	Orders 1, 2 and 3 with Delivered, Shipped and Pending status
Order_Item	3	Products associated with Orders 1, 2 and 3
Payment	3	UPI, Card and COD payment records
INSERT OPERATION	—	Sample customer/product/order records inserted
UPDATE OPERATION	—	Order status updated from Pending/Shipped as applicable
DELETE OPERATION	—	Customer record deleted using customer_id

7.3 Constraints & Data Integrity

List the constraints and validations applied to keep the data accurate and consistent (no duplicate IDs, valid values, referential integrity).

Constraint / Validation	Applied On	Purpose	Evidence / Screenshot
PRIMARY KEY	All main tables	Uniquely identifies records	PK list
FOREIGN KEY	Address, Category, Product, Orders, Order_Item, Payment	Maintains referential integrity	FK list
NOT NULL	Customer, Product, Order_Item etc.	Prevents mandatory fields from being empty	Examples in DDL
UNIQUE	Customer.email, Payment.transaction_id	Prevents duplicate values	PPT constraints
CHECK	Product.price, Order_Item.quantity	Ensures valid ranges	price >= 0; quantity > 0
DEFAULT	Dates/status fields	Supplies default values	CURRENT_DATE / Pending

7.4 SQL Operations & Queries

Include representative examples of DDL, DML, DQL, CRUD, Joins, Aggregate Functions, GROUP BY / HAVING, Subqueries, Views, and other relevant queries.

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
1	DDL-Create Database	Creates the project database.	CREATE DATABASE ECommerce_Order_Management;
2	DDL-Create Table	Creates tables with required constraints.	CREATE TABLE Customer (...);
3	DML-INSERT	Adds new customer records.	INSERT INTO Customer (first_name, last_name, email) VALUES ('Anita','Patel','anita@mail.com');
4	DML-UPDATE	Updates an existing order status.	UPDATE Orders SET order_status='Shipped' WHERE order_id=2;
5	DML-DELETE	Deletes a customer record.	DELETE FROM Customer WHERE customer_id=3;
6	DQL-SELECT	Displays all customer records.	SELECT * FROM Customer;
7	CRUD-CREATE	Creates a new customer record.	INSERT INTO Customer (first_name,last_name,email)

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
			VALUES ('Rahul','Kumar','rahul@mail.com');
8	CRUD-READ	Reads customer information.	SELECT * FROM Customer;
9	CRUD-UPDATE	Modifies order information.	UPDATE Orders SET order_status='Shipped' WHERE order_id=2;
10	CRUD-DELETE	Removes a customer record.	DELETE FROM Customer WHERE customer_id=3;
11	JOIN	Displays customers with their orders.	SELECT c.first_name, o.order_id, o.order_status FROM Customer c JOIN Orders o ON c.customer_id=o.customer_id;
12	Aggregate Function	Counts the number of orders for each customer.	SELECT customer_id, COUNT(*) AS total_orders FROM Orders GROUP BY customer_id;
13	GROUP BY	Groups orders according to customer.	SELECT customer_id, COUNT(*) AS total_orders FROM Orders GROUP BY customer_id;
14	Flitering	Displays products costing more than 1000.	SELECT product_name, price FROM Product WHERE price > 1000;
15	Sorting	Displays products above 1000 in descending price order.	SELECT product_name, price FROM Product WHERE price > 1000 ORDER BY price DESC;
16	VIEW	Creates a view containing order details.	CREATE VIEW Order_Details AS SELECT o.order_id, c.first_name, o.order_date, o.order_status FROM Orders o JOIN Customer c ON o.customer_id=c.customer_id;

127.5 Sample Outputs / Screenshots

Paste important execution screenshots with a short caption. Screenshots should clearly show the query and its output.

Figure 1:

[Insert Screenshot Here]

Figure 2: _____

[Insert Screenshot Here]

Figure 3: _____

[Insert Screenshot Here]

Figure 4: _____

[Insert Screenshot Here]

8. MODULES & FUNCTIONALITIES

Explain the major modules/features of the system and what each module does.

S.No	Module Name	Functionality
1	Customer Management	Stores and manages customer information such as name, email, phone number, and registration date.
2	Address Management	Stores customer addresses including street, city, state, pincode, and address type.
3	Category Management	Maintains product categories and supports parent-child category relationships.
4	Product Management	Stores product details such as product name, description, price, stock quantity, category, and tags.
5	Order Management	Creates and manages customer orders, including order date, status, customer, and delivery address.
6	Order Item Management	Maintains individual products included in an order, including quantity, unit price, and subtotal.
7	Payment Management	Stores payment details such as payment date, amount, payment method, status, and transaction ID.
8	CRUD Operations	Supports Create, Read, Update, and Delete operations for managing database records.
9	SQL Query & Reporting	Retrieves and analyzes data using filtering, sorting, joins, aggregate functions, and views.
10	Data Validation & Integrity	Maintains accurate and consistent data using PK, FK, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints.

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Provide the repository link and clearly mention each team member's contribution.

Repository Link: <https://github.com/poojithakolla-sketch/ecommerce-order-management-system>

Student Name	Roll Number	Role / Responsibility	Contribution Status
K. Poojitha	25B11AI545	ER Diagram, Database Design, Table Creation, PPT Preparation & GitHub Management	Completed
K. Gayathri	25B11AI624	SQL Queries, CRUD Operations & Data Manipulation	Completed
V. Chandrika	25B11AIC65	Sample Data Insertion, Query Testing & Validation	Completed

Student Name	Roll Number	Role / Responsibility	Contribution Status
K. Jeevan	25B11AI553	Project Documentation & Presentation Support	Completed

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

Mention planned or implemented enhancements such as additional modules, security, performance improvements, reporting, UI integration, or scalability.

The following enhancements can be considered for the **E-Commerce Order Management System**:

- **Order Tracking:** Add order tracking and delivery management features.
- **Advanced Reporting:** Generate reports for sales, orders, customers, products, and payments.
- **Analytics:** Add advanced analytics to understand sales and order trends.
- **Automation:** Automate order processing and other e-commerce activities.
- **Additional E-Commerce Features:** Extend the system with more features to support future requirements.
- **Scalability:** Improve the database structure and system capabilities to support a larger number of customers, products, and orders.

The PPT specifically mentions **order tracking/delivery management, advanced reporting and analytics, and additional e-commerce features and automation** as future improvements.

11. CONCLUSION

The E-Commerce Order Management System provides a structured and relational approach to managing online shopping data.

The proposed database contains entities such as Customer, Product, Category, Order, Order Item, Payment, Delivery, and Review.

The relationships between these entities help represent the complete order management process, from a customer placing an order to payment and delivery.

12. REFERENCES

List textbooks, articles, documentation, websites, or other resources referred to during the project.

1. _Oracle Database Documentation
2. Oracle SQL Developer Documentation
3. W3Schools – SQL Tutorial
4. GeeksforGeeks – DBMS and SQL
5. TutorialsPoint – Database Management System

APPENDIX

A. Complete SQL Script

CREATE TABLE

INSERT

SELECT

UPDATE

DELETE

JOIN

GROUP BY / aggregate queries

Constraints and relationships

B. Additional Screenshots / Outputs

Tables created successfully

Sample data

Important SQL queries

Query outputs/results